Google Play Store Market Analysis

February 20, 2026

[20]:
```
# Paso 1
```

[2]:
```
# Importar librerías necesarias
import pandas as pd
import matplotlib.pyplot as plt
```

[3]:
```
# Cargar el dataset apps.csv
apps = pd.read_csv("C:/Users/emata/Desktop/apps.csv")
```

[4]:
```
# Eliminar duplicados (si los hay)
apps = apps.drop_duplicates()
```

[5]:
```
# Imprimir el número total de apps (filas) tras limpiar duplicados
print("Total number of apps in the dataset = ", apps.shape[0])
```

```
Total number of apps in the dataset =  9659
```

[6]:
```
# Imprimir estadísticas descriptivas del dataset
print(apps.describe())
```

```
            Rating       Reviews         Size
count  8196.000000  9.659000e+03  8432.000000
mean      4.173243  2.165926e+05    20.395327
std       0.536625  1.831320e+06    21.827509
min       1.000000  0.000000e+00     0.000000
25%       4.000000  2.500000e+01     4.600000
50%       4.300000  9.670000e+02    12.000000
75%       4.500000  2.940100e+04    28.000000
max       5.000000  7.815831e+07   100.000000
```

[7]:
```
# Vista previa del DataFrame
print(apps.head())
```

```
                                                App        Category  Rating  \
0     Photo Editor & Candy Camera & Grid & ScrapBook  ART_AND_DESIGN     4.1
1                                Coloring book moana  ART_AND_DESIGN     3.9
2  U Launcher Lite - FREE Live Cool Themes, Hide …  ART_AND_DESIGN     4.7
3                               Sketch - Draw & Paint  ART_AND_DESIGN     4.5
4              Pixel Draw - Number Art Coloring Book  ART_AND_DESIGN     4.3
```

```
   Reviews  Size     Installs  Type Price Content Rating  \
0      159  19.0      10,000+  Free     0       Everyone
1      967  14.0     500,000+  Free     0       Everyone
2    87510   8.7   5,000,000+  Free     0       Everyone
3   215644  25.0  50,000,000+  Free     0           Teen
4      967   2.8     100,000+  Free     0       Everyone

                      Genres      Last Updated         Current Ver  \
0               Art & Design   January 7, 2018               1.0.0
1  Art & Design;Pretend Play  January 15, 2018               2.0.0
2               Art & Design    August 1, 2018               1.2.4
3               Art & Design      June 8, 2018  Varies with device
4    Art & Design;Creativity     June 20, 2018                 1.1

    Android Ver
0  4.0.3 and up
1  4.0.3 and up
2  4.0.3 and up
3    4.2 and up
4    4.4 and up
```

[21]:
```
# Paso 2
```

[10]:
```
# Lista de caracteres a eliminar
chars_to_remove = [',', '+', '$']
```

[11]:
```
# Lista de las columnas a limpiar
cols_to_clean = ['Installs', 'Price']
```

[12]:
```
# Loop para cada columna
for col in cols_to_clean:
    # Loop para cada caracter especial
    for char in chars_to_remove:
        # Reemplaza con una función lambda el carácter especial por un texto␣
 ↪vacío ('')
        apps[col] = apps[col].apply(lambda x: x.replace(char, ''))

    # Convierte la columna a tipo flotante (float)
    apps[col] = apps[col].astype(float)
```

[13]:
```
# Revisar tipos de datos después de la limpieza
print(apps[['Installs', 'Price']].dtypes)
```

```
Installs    float64
Price       float64
dtype: object
```

[14]:
```python
# Revisar valores únicos para detectar residuos de caracteres especiales
print("Valores únicos en Installs (muestra):", apps['Installs'].unique()[:10])
print("Valores únicos en Price (muestra):", apps['Price'].unique()[:10])
```

```
Valores únicos en Installs (muestra): [1.e+04 5.e+05 5.e+06 5.e+07 1.e+05 5.e+04
1.e+06 1.e+07 5.e+03 1.e+08]
Valores únicos en Price (muestra): [0.   4.99 3.99 6.99 1.49 2.99 7.99 5.99 3.49
1.99]
```

[15]:
```python
# Comprobar si todas las filas tienen valores numéricos válidos (sin NaNs tras␣
↪la conversión)
print("Installs con NaN:", apps['Installs'].isna().sum())
print("Price con NaN:", apps['Price'].isna().sum())
```

```
Installs con NaN: 0
Price con NaN: 0
```

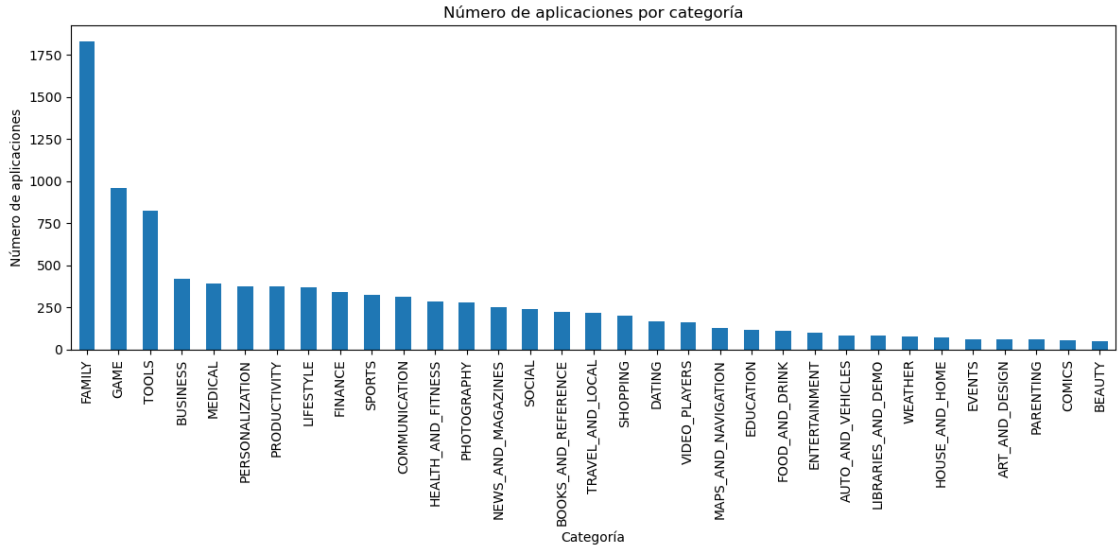
[23]:
```python
# Paso 3
```

[16]:
```python
# Imprime el total de categorías únicas
num_categories = apps['Category'].nunique()
print('Number of categories =', num_categories)
```

```
Number of categories = 33
```

[17]:
```python
# Cuenta el número de aplicaciones en cada categoría y ordena de manera␣
↪descendente
num_apps_in_category = apps['Category'].value_counts()
```

[18]:
```python
# Muestra el resultado en una gráfica de barras
num_apps_in_category.plot(kind='bar', figsize=(12, 6))
plt.xlabel('Categoría')
plt.ylabel('Número de aplicaciones')
plt.title('Número de aplicaciones por categoría')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



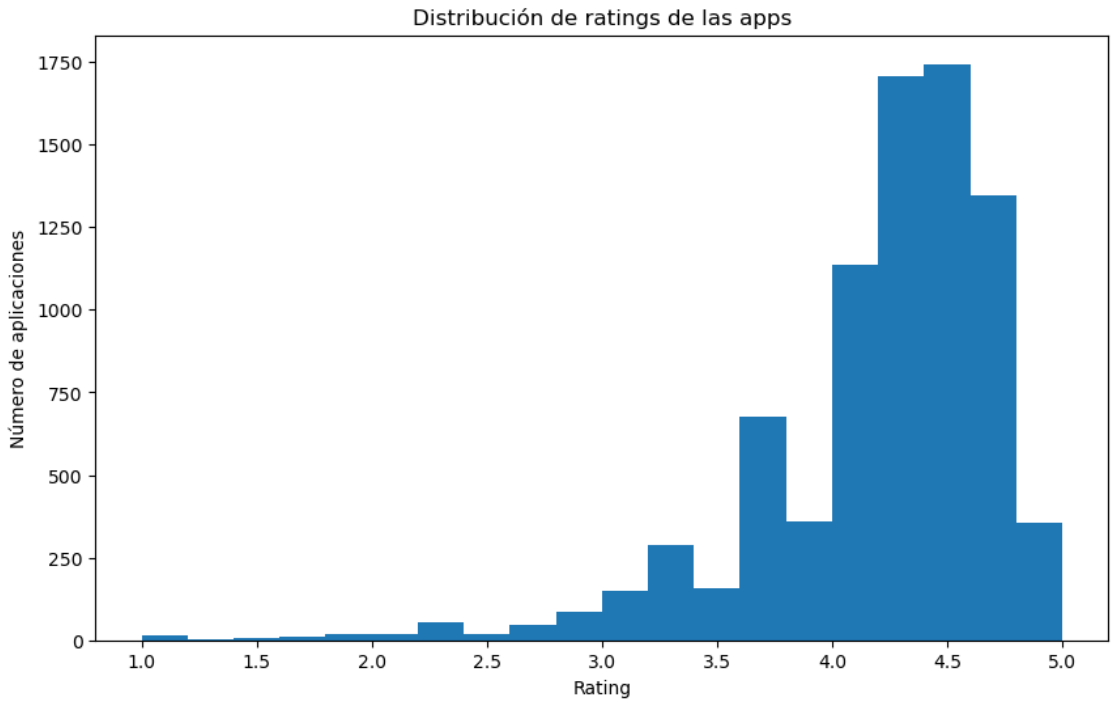
[24]:
```
# Paso 4
```

[25]:
```
# Calcular el promedio de calificación de las apps
avg_app_rating = apps['Rating'].mean()
print('Average app rating =', avg_app_rating)
```

```
Average app rating = 4.173243045387994
```

[26]:
```
# Calcular el promedio de calificación por categoría
avg_rating_per_category = apps.groupby('Category')['Rating'].mean()
```

[27]:
```
# Visualizar en un histograma el comportamiento del Rating
apps['Rating'].hist(bins=20, figsize=(10, 6))
plt.xlabel('Rating')
plt.ylabel('Número de aplicaciones')
plt.title('Distribución de ratings de las apps')
plt.grid(False)
plt.show()
```



[28]:
```
# Paso 5
```

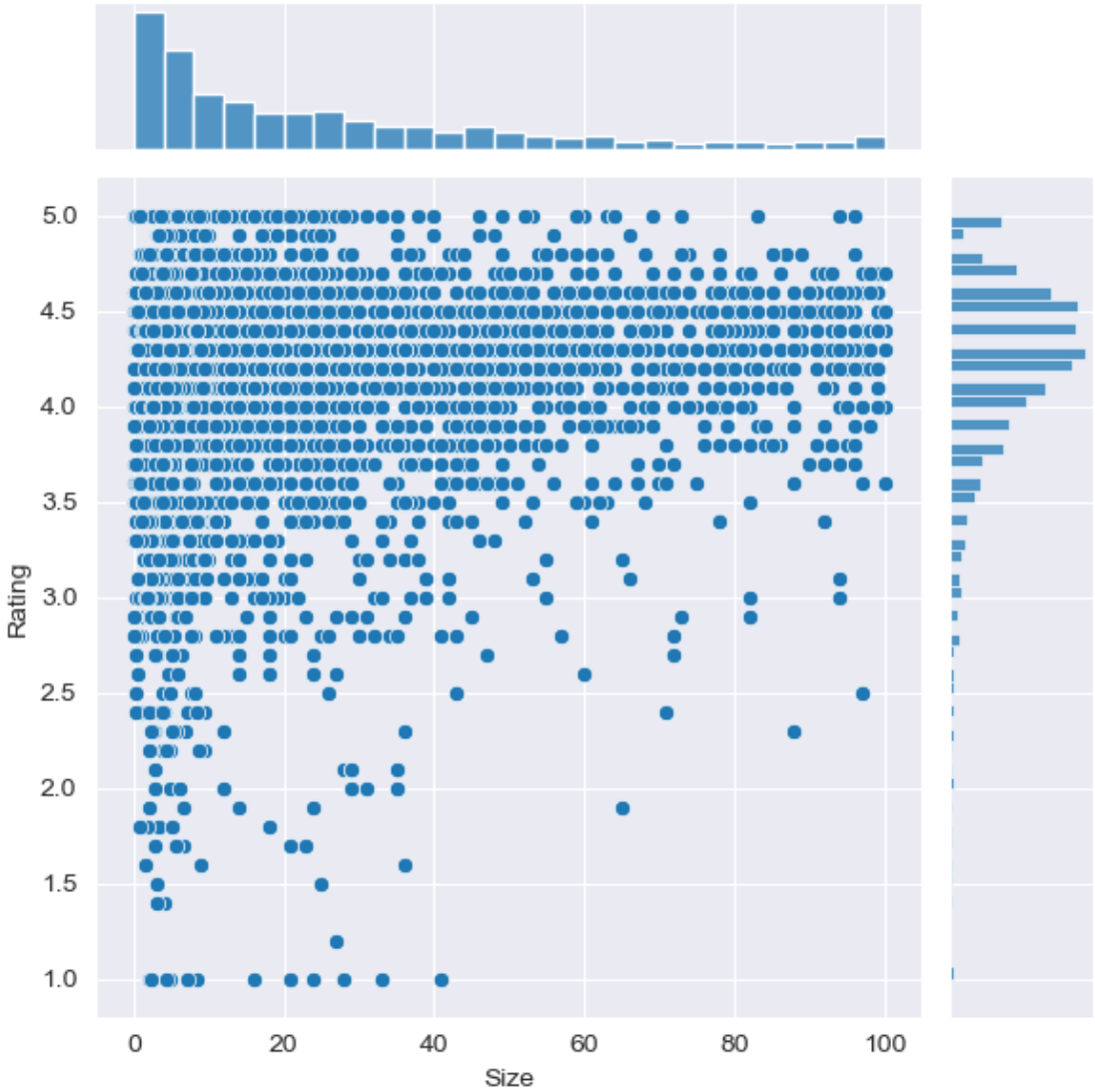
[29]:
```
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

sns.set_style("darkgrid")
```

[30]:
```
# Filtra filas donde los valores de 'Rating' y 'Size' no sean nulos
apps_with_size_and_rating_present = apps[apps['Rating'].notnull() &
 ↪apps['Size'].notnull()]
```

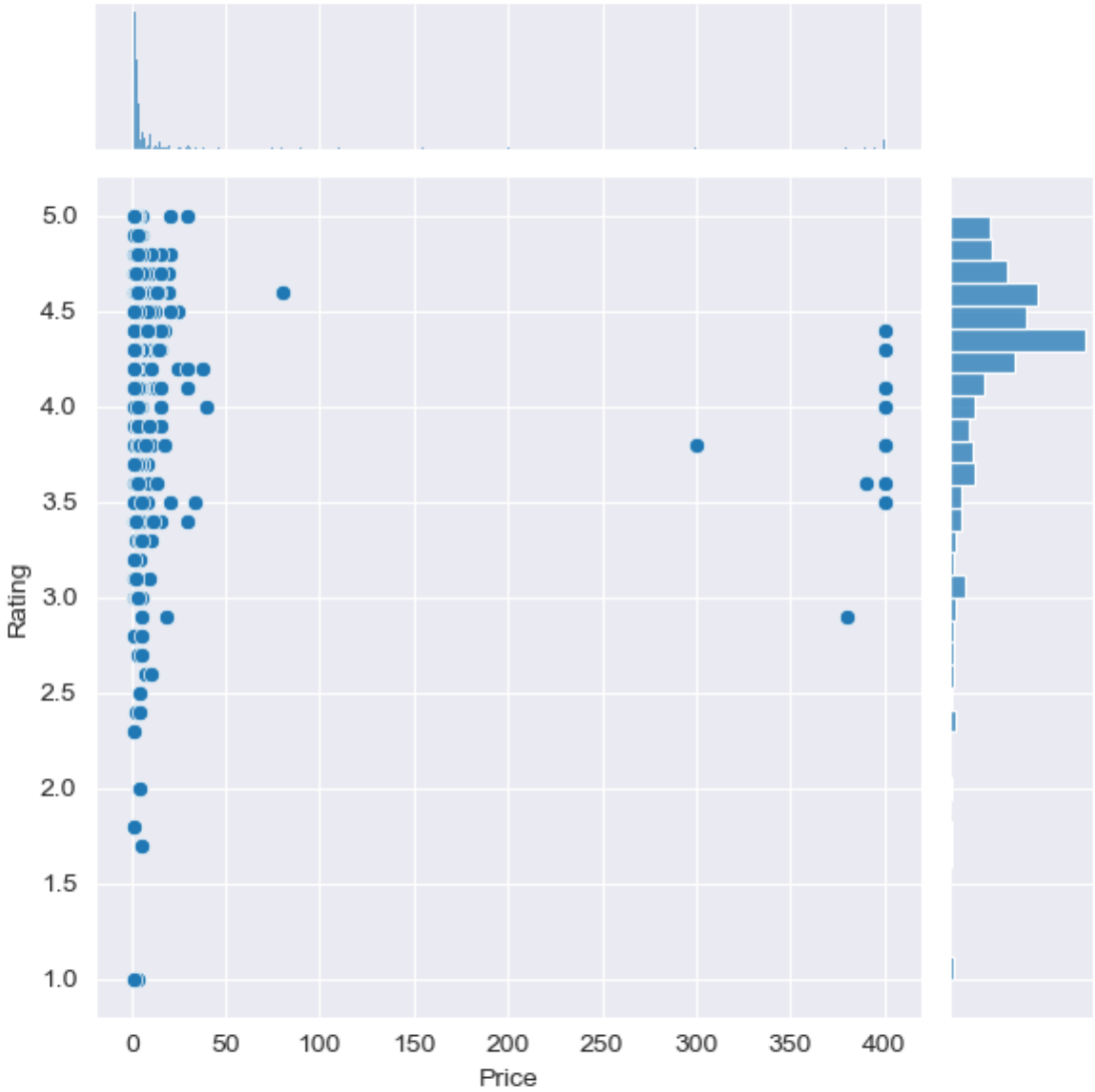
[32]:
```
# Filtra las categorías con al menos 250 apps
large_categories = apps_with_size_and_rating_present.groupby('Category').
 ↪filter(lambda x: len(x) >= 250)
```

[33]:
```
# Gráfica size vs. rating
plt1 = sns.jointplot(x=large_categories['Size'], y=large_categories['Rating'])
```



[34]:
```
# Selecciona las apps de paga 'Type' = 'Paid'
paid_apps = apps[apps['Type'] == 'Paid']
```

[35]:
```
# Gráfica price vs. rating de las aplicaciones de paga
plt2 = sns.jointplot(x=paid_apps['Price'], y=paid_apps['Rating'])
plt.show()
```



[36]:
```
# Paso 6
```

[38]:
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
fig.set_size_inches(15, 8)

# Lista de categorías populares
popular_app_cats = apps[apps.Category.isin([
    'GAME', 'FAMILY', 'PHOTOGRAPHY',
    'MEDICAL', 'TOOLS', 'FINANCE',
    'LIFESTYLE', 'BUSINESS'
])]
```

```python
# Examina la tendencia de precio graficando el Precio por Categoría
ax = sns.stripplot(x=popular_app_cats['Category'], y=popular_app_cats['Price'],␣
↪jitter=True, linewidth=1)
ax.set_title('App pricing trend across categories')

# Selecciona las apps con un precio mayor a 200
apps_above_200 = apps[apps['Price'] > 200]
apps_above_200[['Category', 'App', 'Price']]
```

[38]:

	Category	App	Price
3327	FAMILY	most expensive app (H)	399.99
3465	LIFESTYLE	I'm rich	399.99
3469	LIFESTYLE	I'm Rich - Trump Edition	400.00
4396	LIFESTYLE	I am rich	399.99
4398	FAMILY	I am Rich Plus	399.99
4399	LIFESTYLE	I am rich VIP	299.99
4400	FINANCE	I Am Rich Premium	399.99
4401	LIFESTYLE	I am extremely Rich	379.99
4402	FINANCE	I am Rich!	399.99
4403	FINANCE	I am rich(premium)	399.99
4406	FAMILY	I Am Rich Pro	399.99
4408	FINANCE	I am rich (Most expensive app)	399.99
4410	FAMILY	I Am Rich	389.99
4413	FINANCE	I am Rich	399.99
4417	FINANCE	I AM RICH PRO PLUS	399.99
8763	FINANCE	Eu Sou Rico	394.99
8780	LIFESTYLE	I'm Rich/Eu sou Rico/ /	399.99

[39]:
```
# Paso 7
```

[40]:
```
# Agrupa por tipo de app (Free o Paid) y calcula estadísticas relevantes
apps_by_type = apps.groupby('Type').agg({
    'Rating': 'mean',
    'Size': 'mean',
    'Reviews': 'mean',
    'Installs': 'mean',
    'Price': 'mean'
}).reset_index()

print(apps_by_type)
```

```
   Type    Rating       Size        Reviews      Installs      Price
0  Free  4.166372  20.517555  234243.688532  8.431492e+06   0.000000
1  Paid  4.259603  19.012993    8724.887566  7.587947e+04  14.045146
```

[42]:
```
# Paso 8
```

[43]:
```
# Carga el archivo user_reviews.csv
reviews_df = pd.read_csv("C:/Users/emata/Desktop/user_reviews.csv")
```

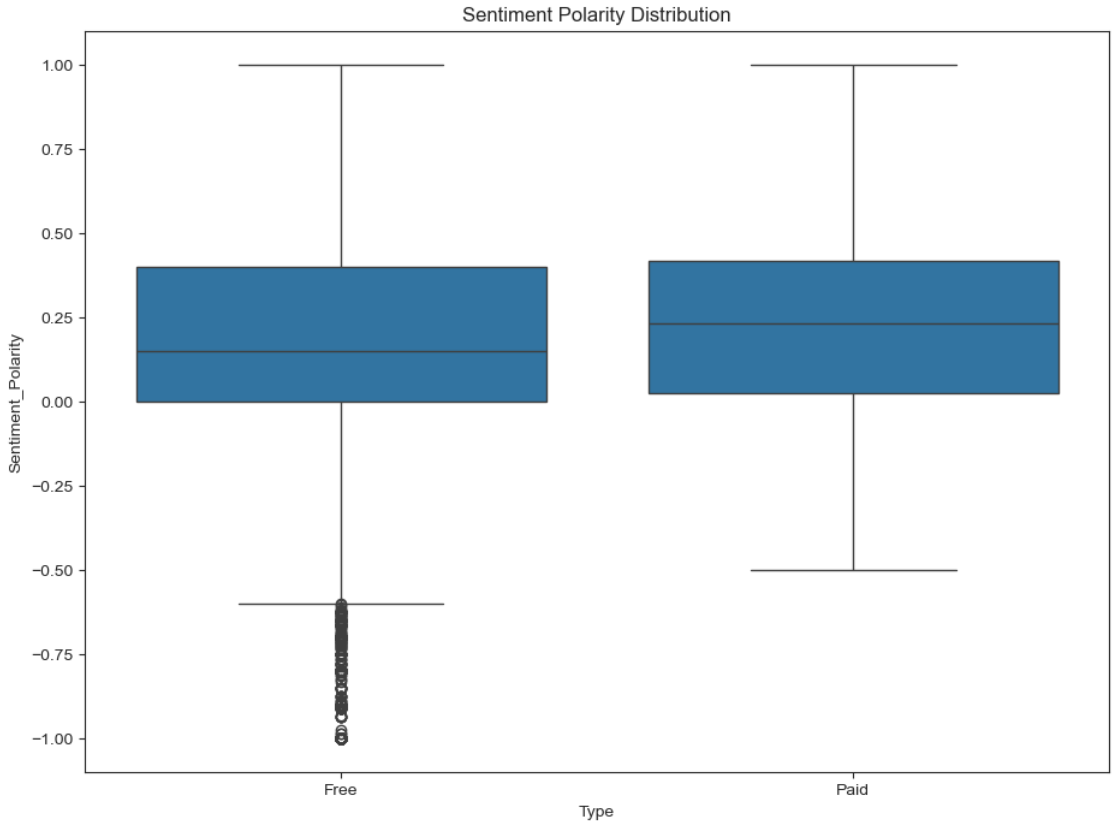
[44]:
```
# Une los dos DataFrames (join)
merged_df = pd.merge(apps, reviews_df, on='App')
```

[45]:
```
# Elimina los valores nulos (NA) de las columnas Sentiment y Review
merged_df = merged_df.dropna(subset=['Sentiment', 'Review'])
```

[46]:
```
# Grafica la polaridad de sentimientos para apps gratuitas y de paga
sns.set_style('ticks')
fig, ax = plt.subplots()
fig.set_size_inches(11, 8)

ax = sns.boxplot(x='Type', y='Sentiment_Polarity', data=merged_df)
ax.set_title('Sentiment Polarity Distribution')
```

[46]:
```
Text(0.5, 1.0, 'Sentiment Polarity Distribution')
```


Sentiment Polarity Distribution
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Sentiment_Polarity
Free
Paid
Type

[ ]: